Operating Systems

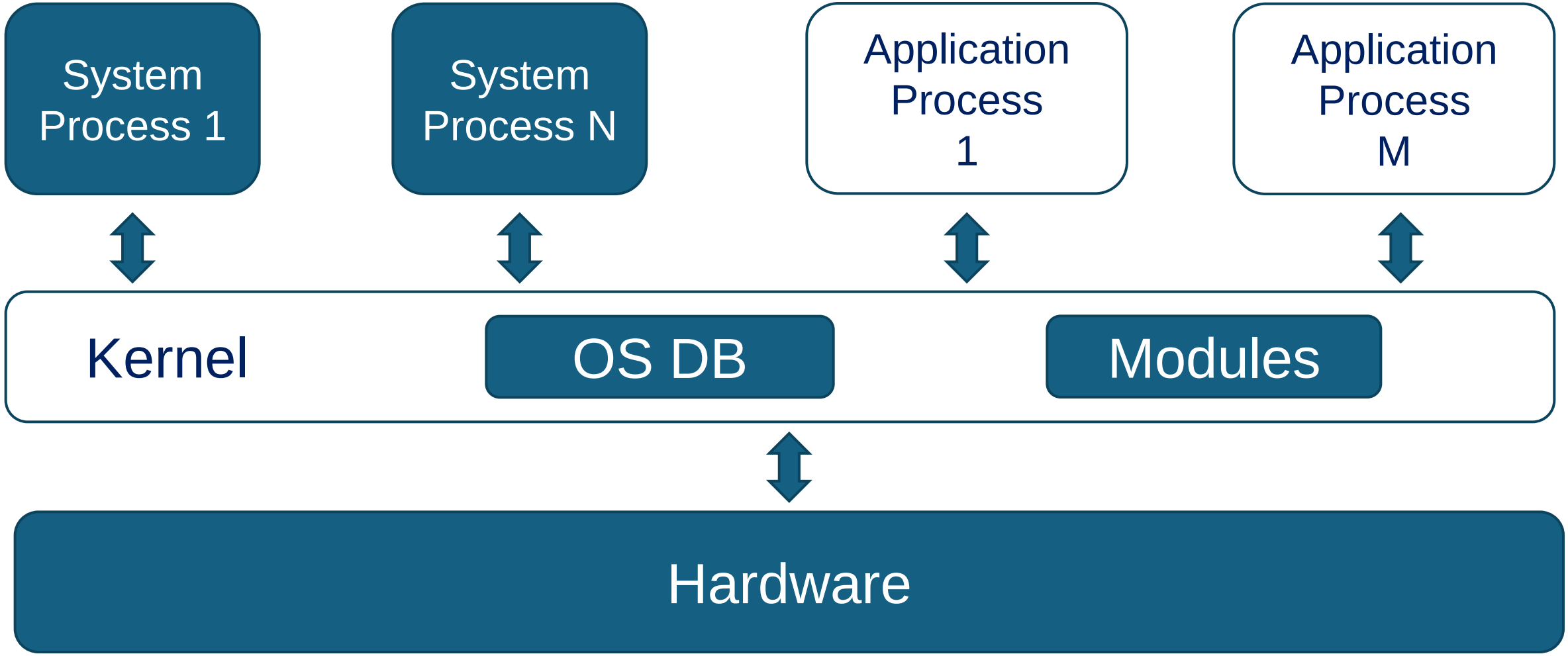
Dmitry Zaitsev

Lecture 17: Configuring and building a customized Linux kernel

Basic components of OS

- **Kernel** – a part of OS which is not a process and, traditionally, resides in RAM, providing basic services, usually process scheduling, inter-process communication, memory control and I/O device control, represented with a series of device drivers
- **System processes** – components, which implement time-consuming service, for instance file operations, network communication, monitoring, planning, etc

Structure of OS



Kernel of OS

- Kernel is entered via interrupts: hardware interrupts – for device drivers and software interrupts – for system call processing
- Kernel consists of modules, which implement operations over resources, and a data base of OS
- Data base of OS represents a multi-linked list of control blocks: process control block, memory page table, device control block, controller control block, etc
- Usually, a uniform entry point is provided for system call implementation – dispatcher of system calls

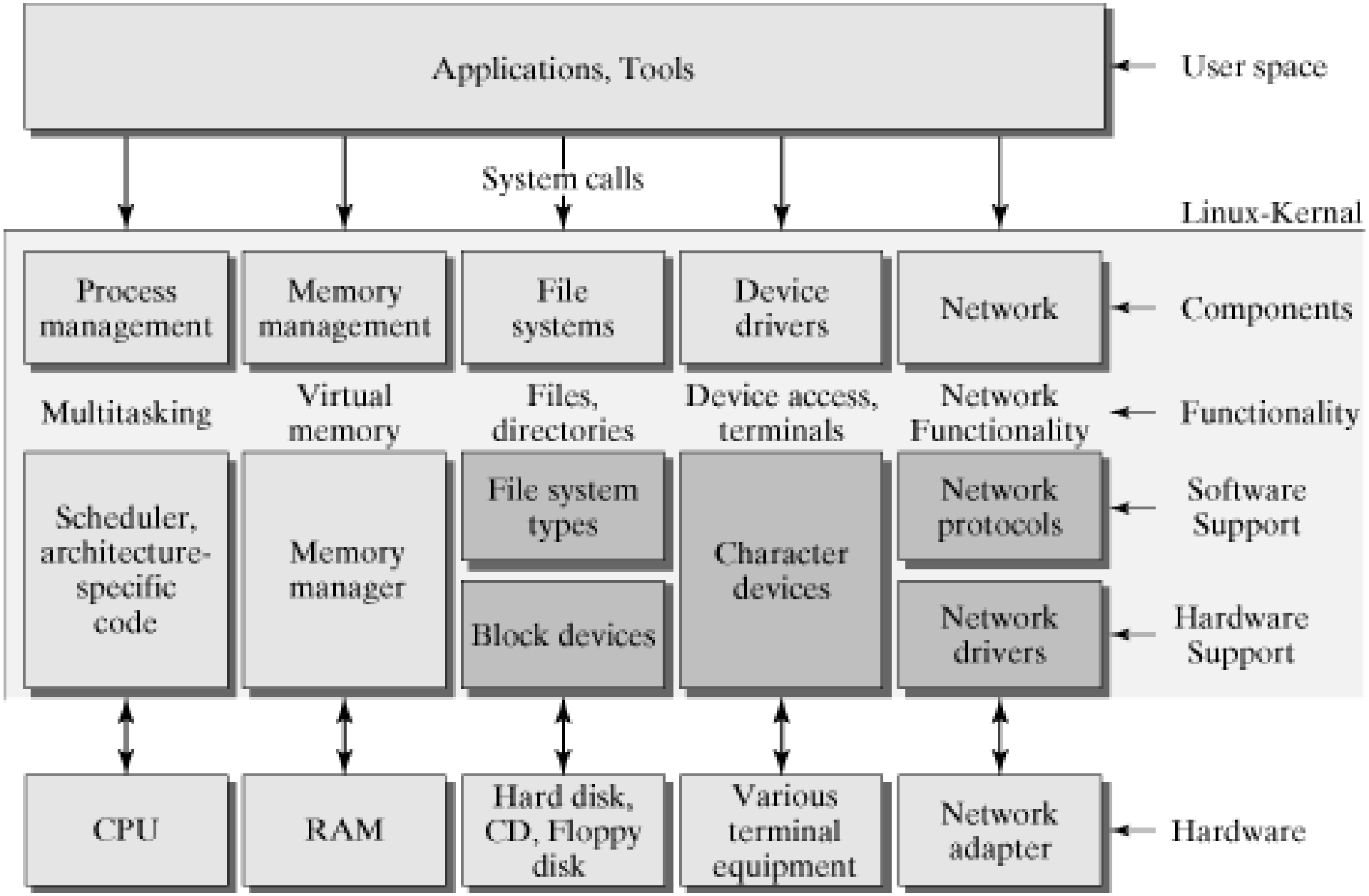
Kernel of Linux. Basic references

- The Linux Kernel Archives https://kernel.org
- Kernel repositories https://git.kernel.org
- Linux Kernel https://github.com/torvalds/linux
- The Linux Kernel documentation https://docs.kernel.org
- Linux resources https://www.linux.com
- Linux kernel news https://lwn.net
- The Linux Tutorial http://www.linux-tutorial.info

Building a custom Linux kernel on Ubuntu

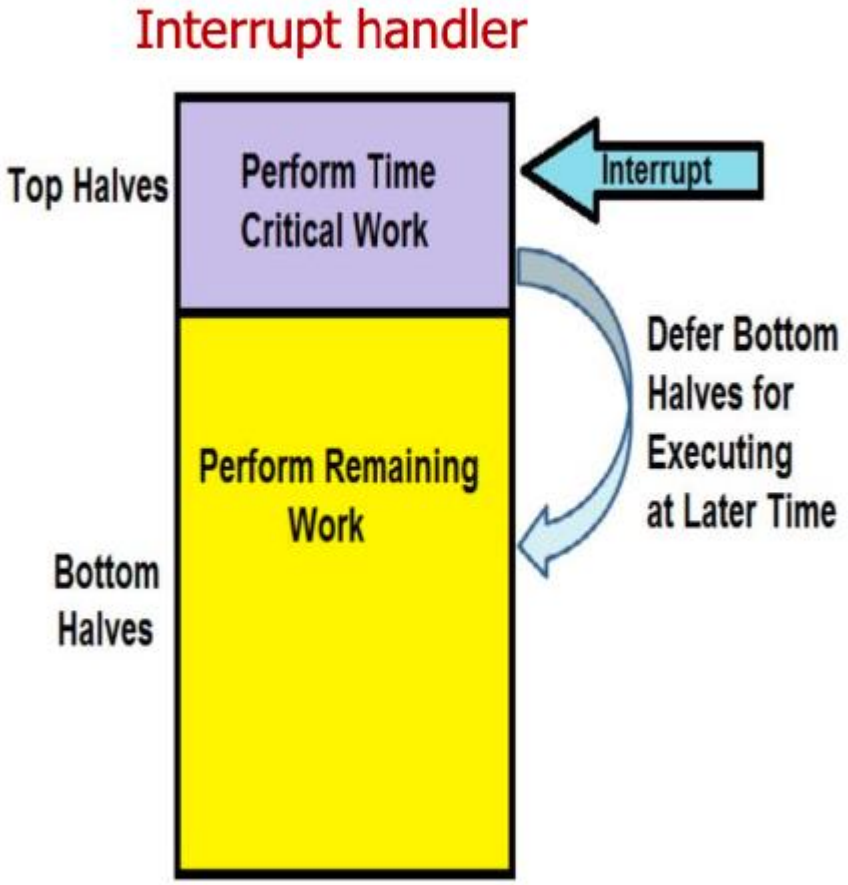
- setting up the environment
- obtaining the source
- configuring the source
- compiling and building images
- configuring boot service
- booting new kernel
- testing new kernel

Linux kernel structure



Activities in Linux Kernel

- kernel threads
- hardware interrupts
- software interrupts
- tasklets
- bottom halves



Interaction Layers

- **User Space.** Application processes and system libraries
- **System Call Interface.** The gateway. Applications send System Calls (open(), read(), socket(), etc) through this layer to request services from the kernel
- **Kernel System Call Handler.** Switches System Calls to corresponding handlers within kernel based on the system call number and specific subfunction. Copies data between kernel and process memory.

Core Kernel

- **Process Scheduler.** Decides which program gets to use the CPU and for how long
- **Memory Manager.** Handles virtual memory, ensuring each process has its own safe space and managing the swap space on disks
- **Virtual File System (VFS).** A universal interface for all file operations. Allows the kernel to implement device I/O and data exchange over network using the same basic commands
- **Network Interface.** Manages all data coming in and out of network connection
- **Inter-Process Communication (IPC).** Interaction between process on the same computer and world-wide

Loadable Kernel Modules (LKMs)

- **.ko (kernel object)** files – modules we can manage manually. They extend the kernel's capabilities dynamically.

Module Category	Examples	Purpose
Device Drivers	GPU, WiFi, Audio, USB	Provides interaction with specific hardware
File Systems	NTFS, ext4, FAT32	Read/write to different types of storage formats
Network Protocols	TCP/IP, Bluetooth	Adds support for different ways of sending data over wires or air
Security	SELinux, AppArmor	Adds extra layers of permission and protection

Install required tools

>sudo apt-get install libncurses5-dev gcc make git exuberant-ctags bc libssl-dev

- ncurses is a library that lets developers create text-based user interfaces (TUIs) in a terminal
- exuberant-ctags is a programming tool that generates tags files to navigate source code
- bc is Basic Calculator to compute in bash expressions over real and other numbers
- libssl-dev is the development package for OpenSSL – basic encryption/decryption

Download and check the latest stable tree

- Download the latest stable tree

>git clone git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git

>cd linux-stable

- Find the latest stable kernel tag

>git tag -l | less

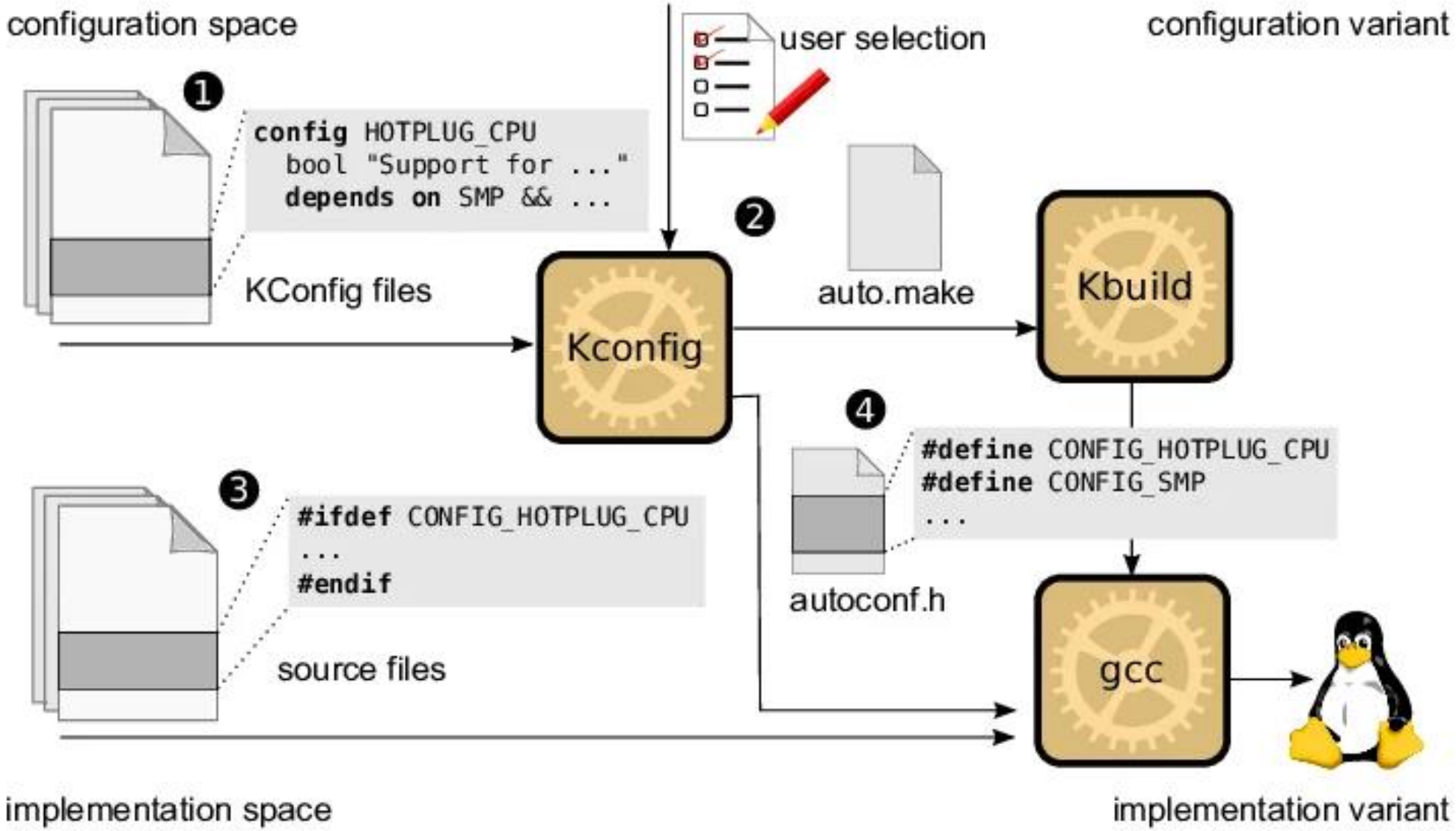
- Checkout the code associated with the kernel

>git checkout -b stable tag

Linux kernel configuration (Kconfig)

- **Kconfig** is the configuration system used by the Linux kernel to define and manage all kernel build options. It describes what features exist in the kernel and how users can enable, disable, or configure them before compiling the kernel.
- Optional features: device drivers, file systems, networking protocols, security features, debugging options
- Kconfig allows developers and users to select which features should be included in the kernel build
- selections are saved in a file .config which is then used by the kernel Makefile system to compile the kernel accordingly

Linux Kernel Configuration and Build Process



Types of Kconfig options

- **Boolean option** can only be **enabled or disabled**. Values:
  - y → built into the kernel
  - n → disabled
- **Tristate option** allows three possible values. Values:
  - y → built into the kernel
  - m → built as a **loadable module**
  - n → not included
- **Text string value** (hostname)
- **Integer value** (buffer size)
- **Hexadecimal number** (physical address)

Menu Symbols

Symbol	Meaning
[ ]	Option disabled
[*]	Built into kernel
<M>	Built as module
< >	Tristate option not selected
---	Menu separator

Dependency symbols

- When a configuration option is visible or selectable based on other configuration options. Ensure that features are enabled only when required prerequisites are satisfied
- **depends** on specifies that a configuration option is only available if another option is enabled
- **select** automatically enables another option when the current option is enabled
- **imply** is a weaker version of select; it suggests enabling another option but does not force it
- options can be conditionally defined using **if**

Symbols of Kconfig

- User-visible Symbols – users can enable, disable, or modify them (CONFIG_USB, CONFIG_NET, CONFIG_SMP)
- Invisible Symbols (Internal Symbols) are not shown to the user in configuration menus, automatically set by dependencies
- Choice Symbols represent mutually exclusive options, where only one option can be selected from a group ("Kernel compression mode“: KERNEL_GZIP or KERNEL_XZ)

Classification of Kconfig symbols

1) Scheduling Symbols. Control the CPU scheduler behavior: choose scheduling algorithm; tune fairness or latency; enable scheduler debugging (CONFIG_SCHED_DEBUG, CONFIG_PREEMPT)
2) Memory Management. Configure how the kernel manages memory (CONFIG_MMU, CONFIG_SWAP)
3) Device Driver. Enable hardware device support (CONFIG_USB_SUPPORT, CONFIG_NETDEVICES)
4) Filesystem Symbols. Configure filesystem: (CONFIG_EXT4_FS, CONFIG_NFS_FS)
5) Networking Symbols. Configure networking stack and protocols (CONFIG_NET, CONFIG_INET, CONFIG_IPV6)

Classification of Kconfig symbols

6) Security Symbols. Control kernel security frameworks and protections (CONFIG_SECURITY, CONFIG_SECURITY_APPARMOR)
7) Power Management Symbols. Control power and energy management (CONFIG_PM, CONFIG_CPU_FREQ)
8) Architecture / CPU Symbols. Configure architecture-specific features (CONFIG_X86, CONFIG_ARM64)
9) Debugging and Tracing Symbols. Enable kernel debugging tools (CONFIG_DEBUG_KERNEL, CONFIG_KGDB)
10) Virtualization Symbols. Support virtual machines and hypervisors (CONFIG_KVM, CONFIG_HYPERV
11) Kernel Infrastructure Symbols Core internal kernel services (CONFIG_SYSFS, CONFIG_NAMESPACES)

Set up your kernel configuration

```
# Duplicate your current config
>cp /boot/config-`uname -r`* .config
# Make the default config
>make defconfig
# Change your config
>make menuconfig
```

Build, install, and run a new kernel

```
# Build the kernel (make –j5)
>make
# Install the kernel
>sudo make modules_install install
# Run your kernel
>sudo update-grub2
```

Kernel configuration interfaces

Command	Interface
>make menuconfig	ncurses terminal menu
>make nconfig	improved terminal UI
>make xconfig	Qt GUI
>make gconfig	GTK GUI
>make config	simple question-by-question

Build kernel for the current configuration

- **Download the Kernel Source**

```
>wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.8.tar.xz
>tar -xf linux-6.8.tar.xz
>cd linux-6.8
```

- **Reuse the configuration from the running kernel**

```
>cp /boot/config-$(uname -r) .config
```

- **(Optional) Modify ConfigurationIf you want to change something:**

```
>make menuconfig
```

- **Build, install, and run kernel**

```
>make -j$(nproc)
>sudo make modules_install
>sudo make install
>sudo update-grub
>sudo reboot
```

Minimal workflow

```
>cp /boot/config-$(uname -r) .config
>make oldconfig
>make -j$(nproc)
>sudo make modules_install
>sudo make install
>sudo update-grub
>reboot
```

Location of built files

Command	File	Location
>make config	Config	/boot/config-<kernel-version>
>make -j$(nproc) >sudo make install	Kernel image	/boot/vmlinuz-<kernel-version>
	Virtual disk initramfs	/boot/initrd.img-<kernel-version>
	Kernel symbol map	/boot/System.map-<kernel-version>
>sudo make modules_install	modules	/lib/modules/<kernel-version>

/boot

Contains files required for booting the system. These files are mainly used by the GRUB bootloader and the Linux kernel:

- Kernel Images. Files that contain the compiled Linux kernel: */boot/vmlinuz-6.8.0-31-generic*
- Initial RAM Disk (initramfs). Temporary root filesystem loaded before the real root filesystem: */boot/initrd.img-6.8.0-31-generic*
- System Map. Kernel symbol table: */boot/System.map-6.8.0-31-generic*
- Kernel Configuration. Configuration used to build the kernel: */boot/config-6.8.0-31-generic*
- GRUB Directory. Bootloader configuration and modules */boot/grub/* Important files inside: /boot/grub/grub.cfg